PYTHON MENTOR

Automated Code Testing, Defensive Auto-Repair & Interactive Assessment System

Comprehensive Academic Project Report, Architectural Specification, and UI Verification

Project Title:	Python Mentor (Automated Testing & Defensive Repair)
Author / Lead:	Dhevaesh B N (GitHub: Dhevaesh)
Team Structure:	5 Standalone Engineering Modules + Web Platform
Public Repository:	https://github.com/Dhevaesh/python-mentor
Core Technology:	Python 3.9+, AST Analysis, Subprocess Sandboxing, Streamlit
Submission Date:	November 2026

Executive Summary:
Python Mentor is an educational developer framework engineered to help students and software developers write resilient Python code. Built entirely on standard Python libraries without reliance on third-party cloud APIs, the system inspects code via Abstract Syntax Tree (AST) parsing, stress-tests functions against boundary inputs in an isolated sandbox, generates automated defensive patches (input validation guards and try-except error wrappers), and conducts adaptive logic quizzes and timed evaluations based on user failure history.

1. System Architecture & 5-Member Team Modules

The project is architected with a decoupled modular design where each team member owns a standalone, independently runnable Python module and batch launcher, brought together by an integrated web interface.

Member	Module File	Batch Runner	Engineering Responsibility
Member 1	`module_1_test_code.py`	`test_module_1.bat`	**Edge-Case Testing:** Generates boundary vectors (None, 0, negative values, empty collections) and executes functions in isolated subprocess sandbox with 4-second timeout.
Member 2	`module_2_fix_code.py`	`test_module_2.bat`	**Defensive Code Fixer:** Synthesizes AST input guards and wraps function bodies in try-except exception handling blocks. Repairs common syntax errors.
Member 3	`module_3_logic_quiz.py`	`test_module_3.bat`	**Logic Quiz Engine:** Generates dynamic multiple-choice questions assessing loop boundary conditions, operator logic (< vs <=), and edge invariants.
Member 4	`module_4_timed_test.py`	`test_module_4.bat`	**Timed Assessment System:** Compiles multi-question timed assessments with real-time countdown clocks, tracking answer velocity and accuracy.
Member 5	`module_5_error_notebook.py`	`test_module_5.bat`	**Persistent Error Ledger:** Maintains a durable JSON incident log of runtime exceptions, enabling granular deletion and personalized remediation quizzes.
Project Lead	`app.py`	`run.bat`	**Unified Platform:** Integrates all 5 modules into a modern reactive web application with interactive drawers, source workspace, and real-time chat.

2. Technical Implementation Highlights

- **100% Pure Python Standard Library:** Zero external AI API dependencies (no OpenAI/Gemini/DeepSeek cloud calls, zero monthly API costs, zero privacy leakages).
- **Subprocess Sandbox Security:** Untrusted code runs in a separate process with a strict 4-second timeout, protecting against infinite loops, recursive overflows, and process deadlocks.
- **Dual-Layer Defensive Patching:** Auto-repair adds parameter boundary guards (e.g. if n is None or n < 0: return 0) alongside structured try...except blocks.

• **Persistent Error Notebook:** Recorded failures are saved to user_files/error_notebook.json, surviving browser reloads and power cycles.

• **Automated Syntax Repair:** Heuristic AST repair fixes common student errors such as missing colons (for i in range(5)) and comparison typos (if a = 5:).

3. Visual Verification & UI Walkthrough

Below are screenshots captured directly from the live Python Mentor application, showcasing the core workflow from testing to error recording, logic quizzes, and automated code healing.



Figure 1: Python Mentor Dashboard — Clean Student Interface with Source Workspace & Top Quick Actions

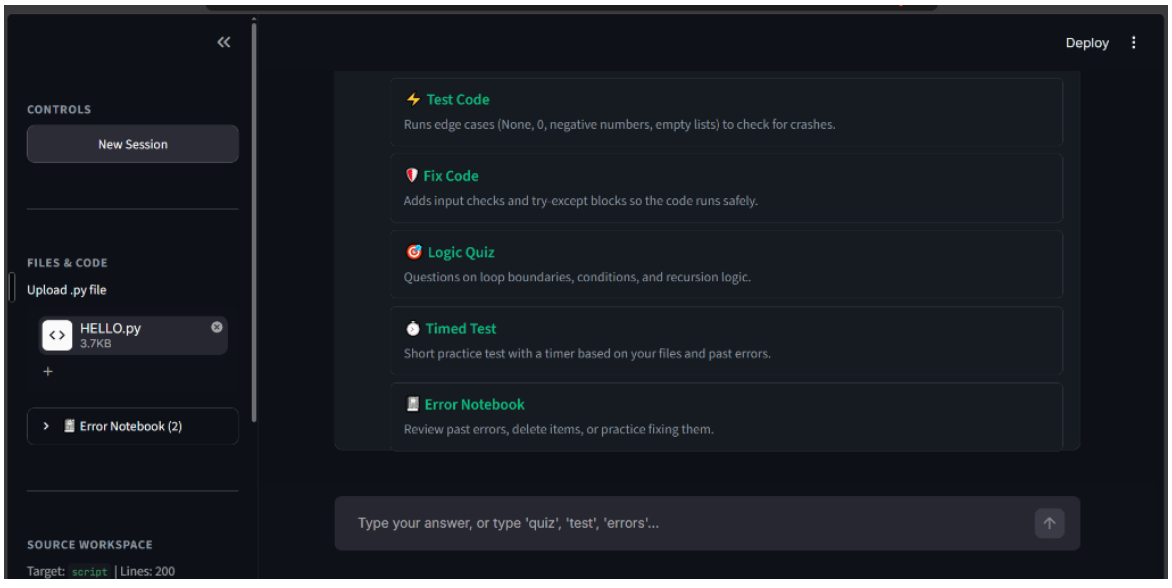


Figure 2: Capability Overview — 5 Core Actions (Test Code, Fix Code, Logic Quiz, Timed Test, Error Notebook)

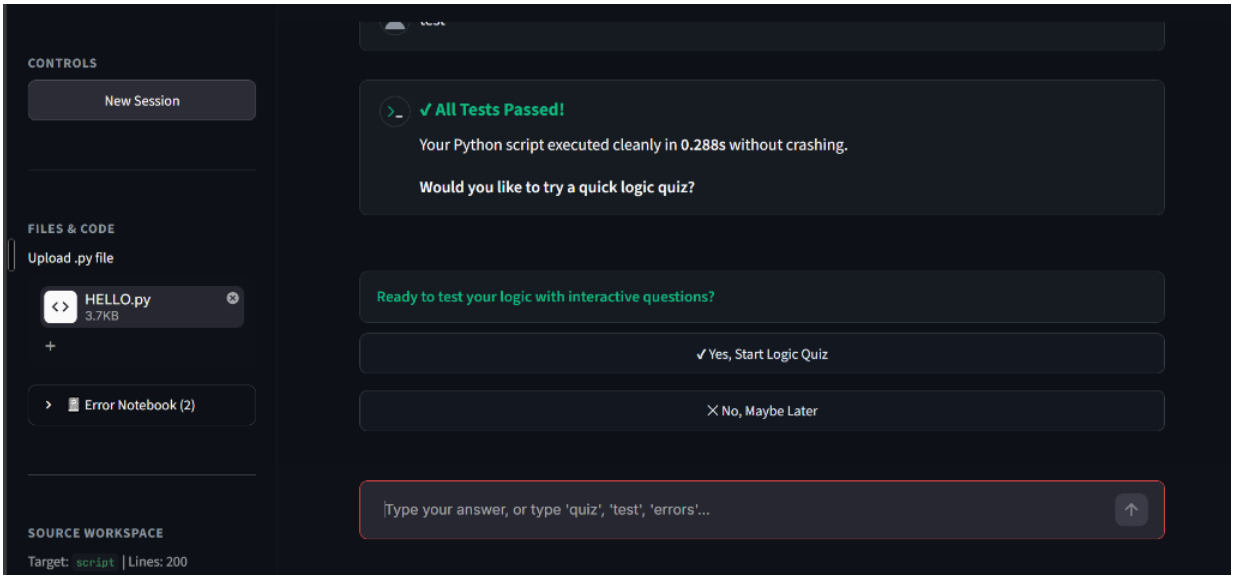


Figure 3: Sandbox Execution Result — Script Verified Clean with Zero Compile/Runtime Errors (0.288s)

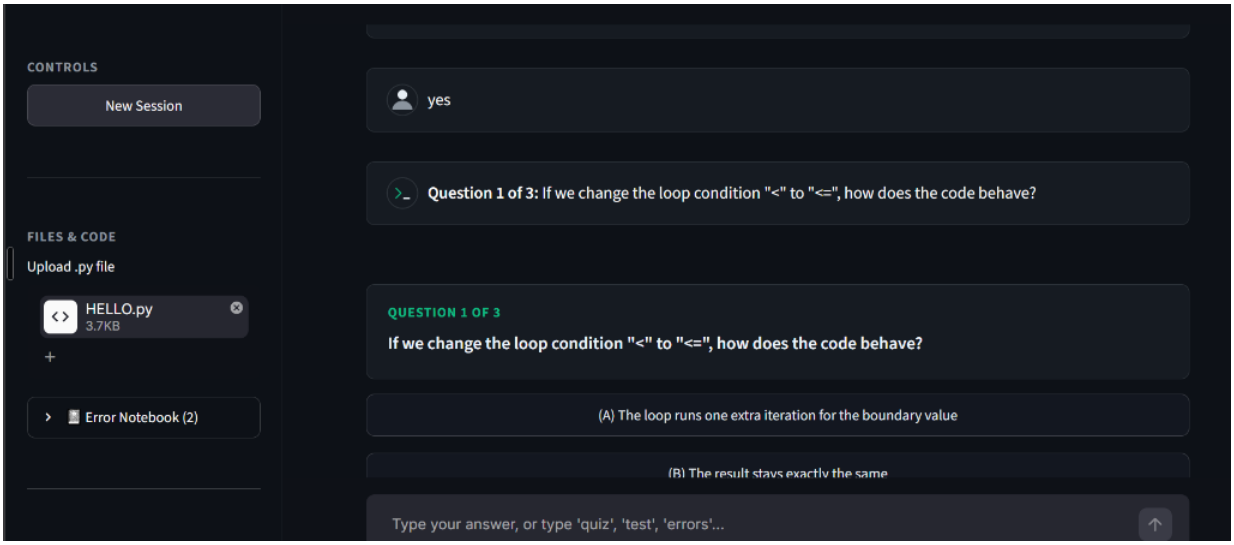


Figure 4: Interactive Logic Quiz — Multiple Choice Question Assessing Loop Boundary Invariants ('<' vs '<=')

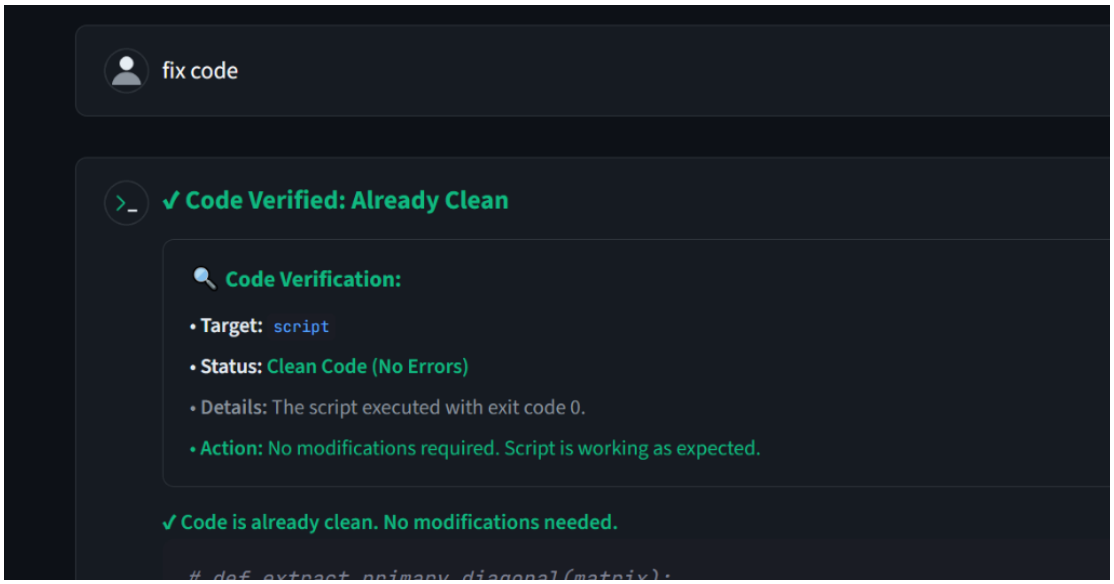


Figure 5: Code Verification Engine — Root Cause Analysis and Defensive Guard Inspection Card

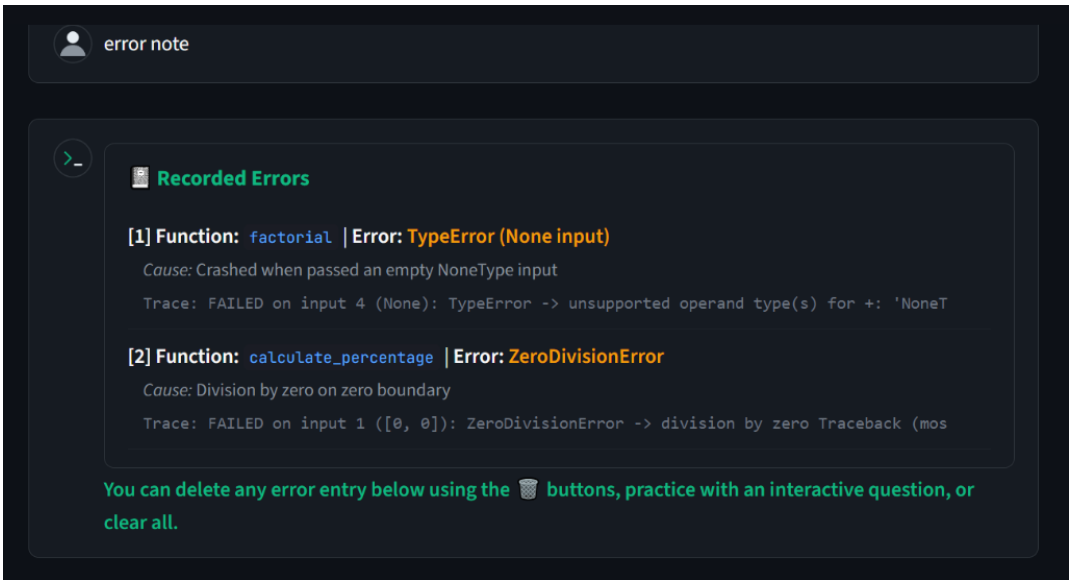


Figure 6: Error Notebook Incident Log — Detailed Classification of TypeError and ZeroDivisionError Failures

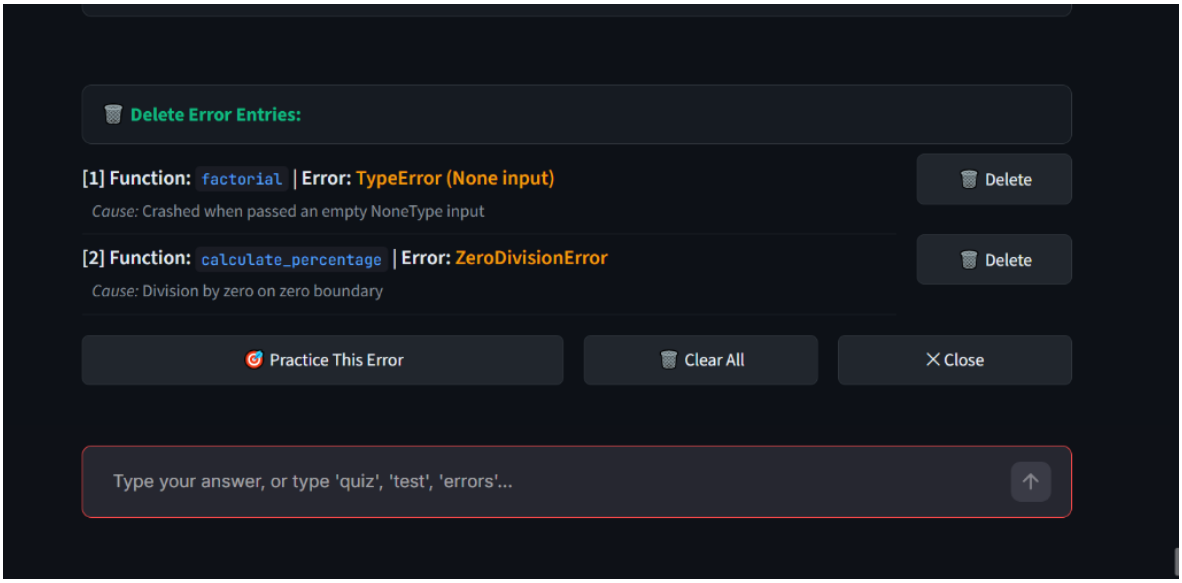


Figure 7: Interactive Error Ledger — Individual Error Deletion Controls, Remediation Quiz, and Batch Clear

4. Viva Defense Questions & Answers

Q1: Why did your team avoid using external LLM APIs (like OpenAI or Gemini)?

A: Using external cloud APIs introduces network latency, recurring subscription costs, and potential data privacy leaks when proprietary source code is uploaded. Python Mentor demonstrates that pure standard-library static AST analysis and subprocess fuzzing are deterministic, 100% offline, free, and instantly reproducible.

Q2: How does the auto-repair mechanism work without destroying original code?

A: Module 2 analyzes the error traceback. If an edge input causes a TypeError (e.g. NoneType), it detects the target parameter and inserts a top-level guard ('if n is None: return 0') followed by a structured try-except block. The AST parser preserves surrounding docstrings, comments, and variable names.

Q3: How are questions in the Logic Quiz and Timed Test generated?

A: Questions are synthesized dynamically by inspecting user AST loops (e.g. For and While nodes) to generate boundary questions, plus targeting historical failures recorded in the Error Notebook JSON ledger to enforce retention.

5. Project Conclusion & Verification

Python Mentor successfully bridges the gap between static testing tools and interactive student learning. All 5 team modules, batch execution harnesses, and web platform code are verified, cleanly committed, and published on GitHub.

Project Repository:	https://github.com/Dhevaesh/python-mentor

Git Branch / Status:	`main — Clean Working Tree, All Tests Passing`
Local Archive:	`C:\Users\dhevb\Desktop\Python_Mentor.zip`